



NAAN MUDHALVAN ARTS INTERNSHIP PROGRAM 2026

Frontend Web Development using React

Project Submission Template

STUDENT NAME: Deepika.P

YEAR: III rd year

COLLEGE CODE: 303

COLLEGE NAME: Arcot Sri Mahaalakshmi Women's college

DEPARTMENT: BCA

SEMESTER: 5th semester

ROLL NO: 30324U09009

NMID: 21cd6b4fd9e9daba4d6429157d289c12

DATE: 23/06/2026

TITLE: Worshop Registration and confirmation app

GITHUB LINK :https://github.com/1308deepikap-cyber/P_Deepika/blob/main/index.html.pdf.pdf

4. PROJECT SCOPE

(Describe what features are included in the application and any limitations or future enhancements that are outside the current scope.)

5. TECHNOLOGIES USED

(Mention all software tools, frameworks, libraries, and technologies used in the development of the project.)

6. APPLICATION FLOW

(Describe how the application works from the user's perspective. Explain the sequence of actions performed by the user and how the system responds. This section should provide a clear understanding of the workflow of the application.)

7. KEY REACT CONCEPTS USED

(Explain the React concepts and features that were used while developing the project. Mention how each concept helped in building the application functionality.)

8. EXPECTED OUTPUT

(Provide the final outcome of the project. Include screenshots of important pages and features.)

9. CHALLENGES FACED

(Describe any difficulties or obstacles encountered during project development and explain how they were resolved.)

10. LEARNING OUTCOMES

(Mention the technical skills, knowledge, and practical experience gained while developing the project.)

11. CONCLUSION

(Summarize the overall success of the project. Explain how the project solved the identified problem and what was achieved through the implementation.)

1.ABSTRACT

Core Entities (Data Models) User/Attendee: Identity, contact details, and registered history. Workshop: Topic, speaker, date, time, total capacity, and price. Registration: Unique ID, timestamps, payment status, and attendance state.

System Architecture Layers

1. Presentation Layer (User Interface) Attendee Portal: Browse events, fill registration forms, and download tickets. Organizer Dashboard: Create workshops, view roster lists, and track revenue metrics.
2. Business Logic Layer (Core Services) Inventory & Booking Service: Checks real-time seat availability. Locks seats temporarily during checkout. Decrements capacity upon successful payment. Payment Service: Interfaces with third-party gateways (Stripe, PayPal). Handles refunds and coupon codes. Notification Service: Triggers instant confirmation emails or SMS. Sends calendar invites (.ics files) and QR codes.
3. Data Layer (Storage) Relational Database: Manages transactional data like user accounts and seat bookings to prevent double-booking. Cache (e.g., Redis): Stores active session states and rapid capacity checks for high-demand events.

Typical Workflow Abstraction

Request: Attendee selects a workshop and submits the registration form. Validation: System verifies seat availability and validates input data. Reservation: System temporarily holds the seat and initiates the payment gateway. Transaction: Payment is processed successfully. Confirmation: System updates database status, finalizes the seat, and dispatches a confirmation email with a unique QR code.

2.PROBLEM STATEMENT

Problem Statement: Inefficient Manual Event Coordination

Organizers face significant administrative burdens when managing workshops using manual or fragmented systems like spreadsheets, generic forms, and separate email clients. This disconnected approach leads to logistical errors, poor user experiences, and lost revenue.

Core Challenges

1. Registration and Inventory Management Overbooking: Lack of real-time seat tracking results in selling more tickets than venue capacity. Cart Abandonment: Holding seats indefinitely during checkout blocks potential paying attendees. Waitlist Chaos: Manually tracking cancellations and filling empty slots is slow and error-prone.
2. Financial and Transaction Friction Payment Tracking: Reconciling manual bank transfers or cash payments with user registrations takes hours. Invoicing: Generating and emailing tax invoices to attendees manually slows down business workflows.

3.PROJECT SCOPE

1. User & Attendee Management**Authentication:** Secure user registration, login, and password recovery.**Profile Management:** View past registrations and download historic receipts.**Guest Checkout:** Allow rapid registration without creating a permanent account.**2. Workshop & Inventory Control****Event Directory:** Search, filter, and view details of upcoming workshops.**Real-time Seat Counter:** Dynamic tracking of remaining tickets to prevent overbooking.**Temporary Seat Locking:** Reserves a selected seat for 10 minutes during checkout.

4.TECHNOLOGIES USED

Framework: React.js or Next.js (Provides a fast, responsive user experience for browsing workshops and filling out forms).**Styling:** Tailwind CSS (Enables rapid, mobile-first UI development).**State Management:** Zustand or Redux Toolkit (Manages seat-locking countdown timers and user session states smoothly).**Backend (Business Logic & APIs)****Runtime Environment:** Node.js with Express.js (Fast, asynchronous architecture ideal for handling concurrent I/O operations).**Alternative Framework:** Python FastAPI or Django (Excellent choice if you plan to integrate data analytics or machine learning reports later).

5.APPLICATION FLOW

Step 1: Discovery: The attendee browses the public event directory and selects a specific workshop.**Step 2: Form Submission:** The user fills out their contact details and clicks "Proceed to Payment."**Step 3: Inventory Check:** The backend executes a database transaction to verify that the remaining seat count is greater than zero.**Step 4: Reservation & Expiry:** The system decrements the available seats temporarily and sets a 10-minute TTL (Time-To-Live) timer in Redis. If the timeout expires without payment, the seat is released back to the public pool.**Step 5: Gateway Handshake:** The backend initializes a secure checkout session with the payment provider (e.g., Stripe) and hands off the user to their payment page.

6.KEY REACT CONCEPT USED

Application Use: Tracks local UI data that changes dynamically based on user interaction.Examples:Storing form inputs (attendee name, email, phone) before submission.Tracking a 10-minute seat-lock countdown timer during checkout.Toggling modal visibility (e.g., displaying the registration terms and conditions).2. Handling Side Effects (useEffect)Application Use: Synchronizes your React components with external systems, APIs, or browser timers.Examples:Fetching the latest workshop details and remaining seat counts from the database on page load.Running a setInterval function to tick down the seat-reservation timer every second.Initializing and cleaning up the mobile camera stream when an organizer opens the QR code scanner.

8.EXCEPTED OUTPUT

Attendee ExperienceWorkshop Catalog: A responsive web page displaying active workshops with real-time seat counters (e.g., "Only 3 seats left!").Checkout Screen: A checkout form equipped with a live 10-minute countdown timer showing how long the seat is locked.Success Screen: A confirmation screen displaying a success checkmark, order breakdown, and a downloadable PDF ticket button.Organizer ExperienceAdmin Dashboard: A private portal showing total revenue, overall registration numbers, and upcoming event lists.Attendee Roster Grid: A searchable table showing each user's registration time, payment status (Pending, Paid, Failed), and check-in status.Mobile-Optimized Scanner UI: A dedicated camera view interface that scans ticket QR codes and immediately updates the screen color to Green (Access Granted), Yellow (Already Checked In), or Red (Invalid Ticket).

9. CHALLENGES FACED

Concurrency and Double-Booking Prevention
The Challenge: If a high-demand workshop has only one seat left, and multiple users click "Buy Ticket" at the exact same millisecond, a standard database update can cause an overbooking anomaly.
The Solution: Implement Database Transactions with Pessimistic Locking (e.g., `SELECT ... FOR UPDATE` in PostgreSQL) or an atomic increment step (`DECR`) in Redis before handing the user over to the payment gateway. This ensures only one process locks and purchases the final slot.

2. Managing the Checkout Race Condition (Ghost Seats)
The Challenge: Users select a ticket, locking it out from the public pool, but then close their browser window without completing the transaction. This leaves "ghost seats" locked up, preventing genuine customers from buying them.
The Solution: Implement a Time-To-Live (TTL) reservation model. When a seat is selected, a 10-minute countdown is tracked in Redis. If a payment webhook is not received within that timeframe, an automated worker releases the seat reservation back into the available pool.

10. LEARNING OUTCOMES

Advanced Database Design and Transactional Integrity
ACID Compliance Mastery: Learning how to use database transactions to prevent race conditions, ensuring that concurrent users cannot book the same final seat.

State Management Isolation: Understanding how to model state transitions (Pending → Paid → Attended) cleanly so that a ticket's status is always verifiable.

Caching & TTL Strategies: Gaining experience with Redis to handle short-lived data, such as a 10-minute temporary checkout seat lock.

2. Mastering Asynchronous System Workflows
Webhook Architecture: Shifting away from fragile client-side redirects and mastering server-to-server webhook communications to handle secure payments.

Background Jobs & Workers: Understanding how to offload heavy tasks—like generating a PDF invoice, creating cryptographic QR codes, and sending emails—to background queues to keep the main user interface fast and responsive.

11.CONCLUSION

Operational Security: Implementing database locks and Redis timeout rules completely eliminates overbooking and "ghost seats," protecting your event revenue. Seamless User Experience: Automated emails, .ics calendar files, and downloadable PDF tickets provide instant confirmation for attendees, while mobile QR code scanning cuts down front-door check-in lines from minutes to milliseconds. Production-Ready Technical Skills: Building this app bridges the gap between basic CRUD applications and complex transactional systems by forcing you to handle real-world challenges like secure third-party payment webhooks, background document creation, and device hardware integration. Ultimately, this architecture serves as a scalable foundation. It can easily expand to support large-scale conferences, multi-session festivals, or global virtual training events.